

API Design Cheat Sheet

System Design Interviews · REST · GraphQL · gRPC · Auth & Security · Patterns

1. Choosing the Right Protocol

Default to REST. Only switch when you have a specific reason. Use the table below to decide quickly.

Protocol	Best For	Signal Words / Triggers	Avoid When
REST	Web & mobile apps, CRUD, public APIs	Default choice — 90% of cases	Almost never avoid REST
GraphQL	Flexible data fetching, multiple client types	"over-fetching", "under-fetching", mobile vs web	Simple CRUD with uniform clients
gRPC / RPC	Internal services, high-performance pipelines	"microservices", "internal API", performance-critical	Public browser clients (needs proxy)
WebSocket	Real-time: chat, live feeds, multiplayer	"real-time", "live updates", "persistent connection"	Simple request/response patterns
SSE	Server-to-client live updates (one-way)	"notifications", "live dashboard", "server push"	When client also needs to push data

Interview tip: Say "I'll use REST APIs here" and move on unless the problem explicitly calls for something else. Interviewers respect decisiveness.

2. REST — The Default Choice

2.1 Resource Modeling

Think about the **things** in your system, not the actions. Resources should be plural nouns. Map your core entities directly to URL paths.

Rules:

- Use plural nouns: /events, /bookings, /users — never /getEvent or /createUser
- Nest resources when the parent is required: /events/{id}/tickets
- Use query parameters for optional filters: /tickets?event_id=123§ion=VIP
- Keep nesting shallow — avoid going deeper than 2 levels

```
# Core entity examples (Ticketmaster-style system)
GET    /events                # List all events
GET    /events/{id}          # Get one event
POST   /events                # Create new event
PUT    /events/{id}          # Replace entire event
PATCH /events/{id}          # Update partial fields
DELETE /events/{id}          # Remove event

# Relationships
```

```
GET    /events/{id}/tickets      # Tickets for this event (parent required)
POST   /events/{id}/bookings     # Create a booking for this event
GET    /tickets?event_id=123   # Optional filter (flat resource)
GET    /events?city=NYC&page=2 # Filter + pagination
```

2.2 HTTP Methods

Each method has a clear purpose. The most important concept to understand is **idempotency**: calling the same request multiple times produces the same result.

Method	Purpose	Idempotent?	Safe?	Example
GET	Retrieve resource(s). Never changes state.	Yes	Yes	GET /events/123
POST	Create a new resource. Server assigns the ID.	No	No	POST /events {body}
PUT	Replace entire resource. Creates if missing.	Yes	No	PUT /events/123 {full object}
PATCH	Update partial fields only.	Maybe	No	PATCH /users/1 {email only}
DELETE	Remove resource. Repeat = same end state (404).	Yes	No	DELETE /bookings/55

Why idempotency matters: Networks fail and clients retry requests. GET, PUT, DELETE are safe to retry. POST is not — two POST calls create two bookings.

2.3 Passing Data to APIs

There are three places to put data in a request. Choosing correctly makes your API intuitive.

Location	Format	Use When	Example
Path parameter	/events/123	Required — identifies the specific resource	GET /events/123/tickets
Query parameter	?city=NYC&page=2	Optional — filtering, sorting, pagination	GET /events?city=NYC&limit=20
Request body	JSON object	Creating or updating — complex/sensitive data	POST /bookings {tickets, payment}

Combined example:

```
POST /events/123/bookings?notify=true
{
  "tickets": [{"section": "VIP", "quantity": 2}],
  "payment_method": "credit_card"
}
```

```
# Path param: 123          (which event — required)
# Query param: notify=true (optional behavior)
# Body:        tickets, payment (the actual data)
```

2.4 HTTP Status Codes

You only need to know the common ones. The most important distinction is **4xx (client error)** vs **5xx (server error)**.

Code	Name	When to Use
200	OK	Standard success for GET, PUT, PATCH
201	Created	POST succeeded — new resource was made
204	No Content	Success but no body to return (DELETE, some PATCH)
400	Bad Request	Malformed syntax, missing required fields
401	Unauthorized	No auth credentials provided (needs to log in)
403	Forbidden	Auth is valid, but no permission for this action
404	Not Found	Resource does not exist
409	Conflict	State conflict — e.g. duplicate booking, optimistic lock
422	Unprocessable Entity	Request is well-formed but fails validation
429	Too Many Requests	Rate limit exceeded — retry after delay
500	Internal Server Error	Bug or crash on the server side
503	Service Unavailable	Server is down or overloaded

3. GraphQL

3.1 What Problem It Solves

REST endpoints return a fixed shape. A mobile app may only need event name and date, while the web dashboard needs full details with venue and ticket data. With REST you either:

- Create multiple endpoints for different use cases (endpoint proliferation)
- Return everything and make clients filter (over-fetching — wastes bandwidth)

GraphQL uses a **single endpoint** that accepts queries describing exactly what data the client wants. The server returns precisely that shape, nothing more.

3.2 Schema Design

You define types and their relationships once. Clients can traverse the graph in a single query.

```
type Event {
  id:      ID!
  name:    String!
  date:    DateTime!
```

```

    venue:    Venue!           # Nested type – traversable
    tickets: [Ticket!]!       # List of tickets
}

type Venue {
    id:        ID!
    name:      String!
    address: String!
}

type Query {
    event(id: ID!): Event
    events(limit: Int, after: String): [Event!]! # Cursor pagination
}

type Mutation {
    createBooking(input: BookingInput!): Booking!
    cancelBooking(id: ID!): Boolean!
}

type Subscription {
    ticketSold(eventId: ID!): TicketEvent! # Real-time updates
}

```

3.3 Query Examples

The same endpoint handles all queries. Each client requests exactly what it needs:

```

# Mobile: only name and date
query GetEventMobile {
    event(id: "123") {
        name
        date
    }
}

# Web: full details in one round trip
query GetEventWeb {
    event(id: "123") {
        name
        date
        venue { name address }
        tickets { section price available }
    }
}

```

```

}

# Mutation
mutation CreateBooking {
  createBooking(input: { eventId: "123", tickets: [{section: "VIP", qty: 2}] }) {
    id
    status
  }
}

```

3.4 The N+1 Problem (Most Important GraphQL Gotcha)

When you query 100 events with their venues, a naive implementation fires 101 database queries: 1 for events, then 1 per event for its venue. This destroys performance at scale.

```

# Problem: 100 events = 101 queries
# 1 query:  SELECT * FROM events LIMIT 100
# 100 more: SELECT * FROM venues WHERE id = ? (once per event)

# Solution: DataLoader batches related queries
class VenueLoader(DataLoader):
    async def batch_load_fn(self, venue_ids):
        # 1 query: SELECT * FROM venues WHERE id IN (...all ids...)
        venues = await db.query('SELECT * FROM venues WHERE id = ANY($1)',
[venue_ids])
        return [venues_by_id.get(id) for id in venue_ids]

```

3.5 When to Use GraphQL

Situation	Use GraphQL?
Mobile app needs different data than web dashboard	Yes — classic use case
Frontend team iterates without backend involvement	Yes — they request new fields freely
Interviewer says "over-fetching" or "under-fetching"	Yes — direct signal
Simple CRUD app with uniform clients	No — REST is simpler
You need simple HTTP caching (CDN, browser cache)	No — REST GET is cacheable; GraphQL POST is not
Public API for third-party developers	No — REST is more familiar and documented

4. gRPC / RPC

4.1 How It Differs from REST

REST is **resource-oriented**: you model nouns (events, bookings) and use HTTP verbs on them. RPC is **action-oriented**: you call functions across a network as if they were local.

Aspect	REST	gRPC
Paradigm	Resource-oriented (nouns)	Action-oriented (functions)
Protocol	HTTP/1.1 + JSON	HTTP/2 + Protocol Buffers (binary)
Performance	Moderate	High (binary, compressed, multiplexed)
Browser support	Native	Requires grpc-web proxy
Contract	OpenAPI / informal	Strict .proto file — required
Code generation	Optional	Required — generated clients in any language
Streaming	Workarounds (SSE, WebSocket)	Native: unary, server, client, bidirectional
Best for	Public APIs, web & mobile clients	Internal service-to-service communication

4.2 Protocol Buffers (.proto file)

You define your service contract in a .proto file. gRPC generates type-safe client and server code for any language from this single definition.

```
syntax = "proto3";

service BookingService {
    // Unary: one request, one response (like REST)
    rpc CreateBooking (BookingRequest) returns (BookingResponse);

    // Server streaming: one request, many responses
    rpc StreamAvailability (EventId) returns (stream TicketUpdate);

    // Bidirectional streaming: many requests, many responses
    rpc Chat (stream Message) returns (stream Message);
}

message BookingRequest {
    string event_id = 1;    // Field numbers are permanent — never reuse
    string user_id = 2;
    int32 quantity = 3;
    string section = 4;
}

message BookingResponse {
    string booking_id = 1;
    string status = 2;
```

```
}
```

4.3 When to Use gRPC

- Interviewer mentions microservices or internal service communication
- Performance is explicitly a constraint — binary encoding is significantly faster
- Multiple programming languages in the same system (polyglot)
- You need streaming between services (live price updates, log streaming)
- You want compile-time type safety across service boundaries

In practice: Use REST for public endpoints consumed by web/mobile clients. Use gRPC for internal service-to-service calls where you control both sides.

5. Authentication & Authorization

5.1 The Difference

Authentication — Who are you? Verifying identity (logging in, presenting credentials).

Authorization — What are you allowed to do? Checking permissions after identity is confirmed.

Always authenticate first, then authorize. They are separate steps.

5.2 JWT Tokens

Best for user-facing web and mobile applications. A JWT is a self-contained token that encodes the user's identity and permissions. Any service with the verification key can validate it without a database lookup.

```
# JWT structure: header.payload.signature (base64url encoded)

# Payload contains user context:
{
  "user_id": "123",
  "email":   "john@example.com",
  "role":    "customer",
  "exp":     1734307200          # Expiry timestamp
}

# Sent with every request:
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoiaMTIzIn0.abcl23

# Server validates:
# 1. Signature is valid (not tampered)
# 2. Token is not expired
# 3. Claims match the required permissions
```

Key properties of JWT: Stateless — no database lookup needed per request. Short-lived (15–60 min) + refresh token for new access tokens. Any service can verify with the public key.

5.3 API Keys

Best for server-to-server communication and third-party developer access. API keys are long random strings that identify an application, not a user.

```
# Client sends key in header:
GET /events
Authorization: Bearer sk_live_abc123def456...

# Server verifies:
SELECT client_id, permissions, rate_limit
FROM api_keys
WHERE key_hash = hash(received_key)
AND revoked = false
```

When NOT to use API keys: Never for end users — users should not manage cryptographic strings. API keys carry no user context, have no expiry by default, and cannot represent individual users.

5.4 Role-Based Access Control (RBAC)

Assign roles to users, assign permissions to roles. In interviews, note which roles can access which endpoints.

Endpoint	Customer	Manager	Admin
GET /events	Yes — all	Yes — all	Yes
POST /events	No	Yes — own venue	Yes
GET /bookings/{id}	Own only	Yes — all	Yes
DELETE /bookings/{id}	Own only	No	Yes
GET /reports/revenue	No	Own venue only	Yes

6. Pagination

Always include pagination on list endpoints. Returning millions of records in one response is a design failure. Use one of these three approaches:

Type	How It Works	Pros	Cons	Use When
Offset	?offset=40&limit=20	Simple. Easy 'jump to page N'.	Breaks with concurrent inserts. Slow on large offsets.	Simple admin dashboards, no real-time data
Cursor	?cursor=eyJpZCI6NDB9&limit=20	Stable. Fast. Handles inserts gracefully.	Cannot jump to page N. Cursor must be opaque.	Feeds, timelines, high-volume data
Keyset	?after_id=123&after_date=2024-01-01	Most performant on large tables.	Requires composite unique sort key.	Large datasets, production systems

Cursor response shape:

```

{
  "data": [ { "id": 1, ... }, { "id": 2, ... } ],
  "pagination": {
    "next_cursor": "eyJpZCI6MjB9",    # Encode the last-seen ID/timestamp
    "has_next":    true,
    "total_count": 1547
  }
}

# Next page request:
GET /events?cursor=eyJpZCI6MjB9&limit=20

```

7. API Versioning

APIs change over time. Versioning lets you evolve your API without breaking existing clients.

Strategy	Format	Pros	Cons
URL Path (recommended)	/v1/events → /v2/events	Explicit, easy to route, easy to test in browser	URL changes; clients must update
Header	API-Version: 2	Clean URLs, follows HTTP standards	Invisible in URL, harder to test, less common
Query Parameter	/events?version=2	Simple to add without new routes	Pollutes query string, easy to forget
Content Type	Accept: application/vnd.api.v2+json	Purist REST approach	Very complex, rarely used

Interview guidance: URL versioning is the safest choice — most interviewers know it. Versioning is often skipped entirely in interviews, which is fine.

8. Rate Limiting

Protects your system from abuse, scraping, and accidental overuse. Implement at the API gateway level.

8.1 Common Strategies

Strategy	How It Works	Best For
Fixed Window	Count requests per fixed time window (e.g., 1000/hour). Resets at boundary.	Simple implementation, predictable
Sliding Window	Rolling window of the last N seconds. No burst at boundary.	Smoother limiting, more accurate
Token Bucket	Bucket refills at fixed rate. Allows controlled bursts up to bucket size.	Allows legitimate traffic bursts
Leaky Bucket	Requests queue and are processed at a fixed rate. Smooths bursty traffic.	Strict, even output rate

8.2 Response Headers & Status Code

```
HTTP/1.1 429 Too Many Requests
X-RateLimit-Limit:    1000          # Your limit
X-RateLimit-Remaining: 0           # How many left
X-RateLimit-Reset:    1672531200    # Unix timestamp when limit resets
Retry-After:          60            # Seconds until retry is safe
```

Typical limits: 1000 requests/hour per authenticated user • 100 requests/hour per IP (unauthenticated) • 10 requests/minute for sensitive endpoints like POST /bookings

9. Security Checklist

9.1 Input Validation

- Validate and sanitize all input — type, length, format, range
- Reject unexpected fields — do not pass raw input to your database
- Validate on the server — never trust client-side validation alone

9.2 Transport & Data

- Always use HTTPS — never expose endpoints over plain HTTP
- Never log sensitive data — no passwords, card numbers, tokens in logs
- Hash API keys before storing — never store plain text secrets

9.3 Auth & Access

- Validate JWT signature and expiry on every request
- Never trust user-provided IDs without checking ownership (e.g. GET /bookings/456 — does the caller own booking 456?)
- Apply the principle of least privilege — grant minimum permissions needed
- Use short-lived access tokens (15–60 min) and refresh tokens

9.4 Infrastructure

- Rate limit all public endpoints to prevent abuse and DDoS
- Set CORS headers to restrict which origins can call your API
- Return generic error messages to clients — do not expose stack traces or internal details

10. Real-Time Patterns

Standard REST APIs are request/response. When you need the server to push data, you need a different approach.

Pattern	Direction	Protocol	Best For	Complexity
WebSocket	Both (full-duplex)	ws://	Chat, games, collaborative editing	High
SSE	Server → Client	HTTP text/event-stream	Live notifications, dashboards, feeds	Low
Long Polling	Server → Client	HTTP	Simple notifications, legacy support	Medium
Short Polling	Client polls server	HTTP	When real-time is a nice-to-have	Lowest

```
# Server-Sent Events (SSE) — simplest real-time pattern

# Server (Node.js):
res.setHeader("Content-Type", "text/event-stream");
res.setHeader("Cache-Control", "no-cache");
setInterval(() => {
  res.write(`data: ${JSON.stringify({ price: getPrice() })}\n\n`);
}, 1000);

# Client:
const es = new EventSource("/prices/AAPL");
es.onmessage = (e) => updateDisplay(JSON.parse(e.data));
```

11. Interview Quick Reference

Time Allocation

Spend **at most 5 minutes** on API design. Show judgment and move on. Interviewers lose marks for candidates who get bogged down here instead of discussing architecture.

What to Cover (in order)

- State your protocol choice: "I'll use REST APIs here"
- List your key endpoints — 3 to 6 is enough
- Mention authentication: "endpoints are secured with JWT"
- Mention pagination on list endpoints
- Move on to high-level design

What to Skip (unless asked)

- Exact status codes — say '2xx for success, 4xx for client error'
- Versioning — only mention if the interviewer raises it
- Detailed rate limiting algorithms
- Complete request/response schemas

Signal Words and Your Response

If the interviewer says...	Your response
"over-fetching" or "under-fetching"	Mention GraphQL and explain why it fits
"microservices" or "internal service communication"	Consider gRPC for those internal calls
"real-time", "live", "notifications"	Mention WebSocket or SSE and explain the tradeoff
"millions of users" or "scale"	Ensure pagination, rate limiting, and auth are mentioned
"mobile and web clients need different data"	GraphQL is the right answer here
"third-party developers" or "public API"	REST with API keys, versioning, and rate limiting

